Two related methods were added to the Task class: part_of_velocity? (implied by the be_part_of_velocity matcher) and points_toward_velocity. According to the requirements, the part_of_velocity? method returns true if the task has been completed in the last three weeks. As a matter of code style, you're naming the method and testing the behavior rather than testing against the specifics of the implementation. By testing against the behavior, hopefully you'll be better able to deal with the inevitable requirements changes.

The points_toward_velocity method is trickier. If the task counts toward velocity, then the size of the task is returned; otherwise the method returns zero. This is an example of designing the class interface via tests. The idea is to keep all the logic for tasks inside the Task class. Specifically, I want a project to be able to determine how much time is remaining without having to query the task twice—once to determine the status of the task and again to determine its size.

There are few interesting design considerations in the specs themselves. Although the order in which you write the specs probably doesn't make a long-term difference, the order in the file above is what you get if you try to write a spec that will break the existing code each time. By that I mean that I wrote the first test and passed it with very basic changes: a part_of_velocity? method that returned false and a points_toward_velocity method that returned 0. Since the third spec has the same outcome, it would pass without further code changes, requiring larger code changes when the remaining spec is added. However, the second spec, where the task is a part of velocity, requires you to add logic specifying that the task must be complete to count toward velocity. Then the third spec says it must have been completed recently. When you think about writing the next spec so that it breaks existing code, it is easier to move in small steps.

As a matter of testing style, notice the way dates are specified on lines 10 and 16 of the test file. I'm using the Rails helpers to concisely specify the dates relative to the current time: 6.months.ago for the out-of-velocity task and 1.day.ago for the in-velocity task. If I had specified an explicit date for yesterday, then eventually the passage of time would push that date beyond the three-week threshold and the test would fail. Using relative dates reduces that problem.

There's a different interesting question about the test design and the dates. Neither six months nor one day is particularly close to the boundary between in-velocity and out-of-velocity tasks. Shouldn't I test more days or test something closer to the boundary? This question reflects the difference between testing as a design aid and testing for verification. In strict TDD you would avoid writing a test that you expect to pass, because a passing test doesn't normally drive you to change the code.